

INHERITANCE MAPPING IN ORM

148

148

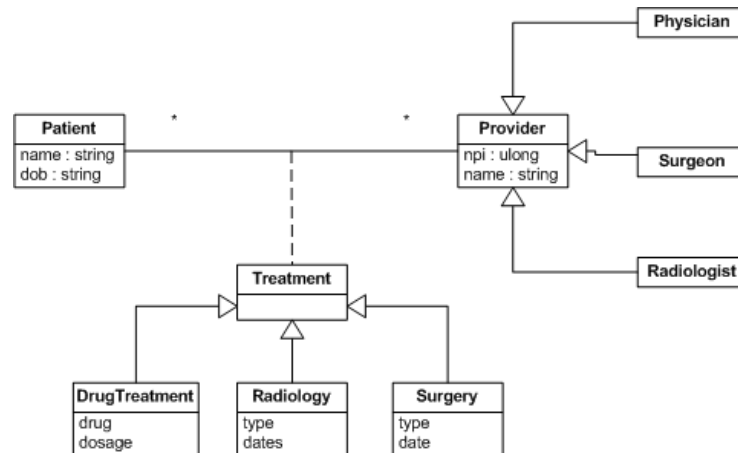
Inheritance Mapping

- Entity Mapping
 - Map entity objects to rows in tables
 - What about objects in subclasses?
- Three Strategies:
 - Single table
 - Joined table
 - Table per concrete subclass

149

149

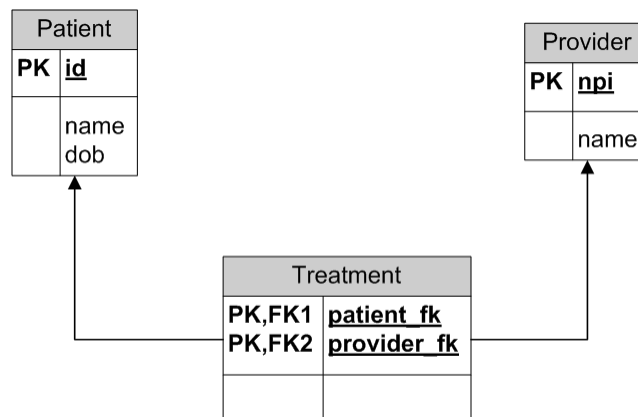
Example: UML Diagram



150

150

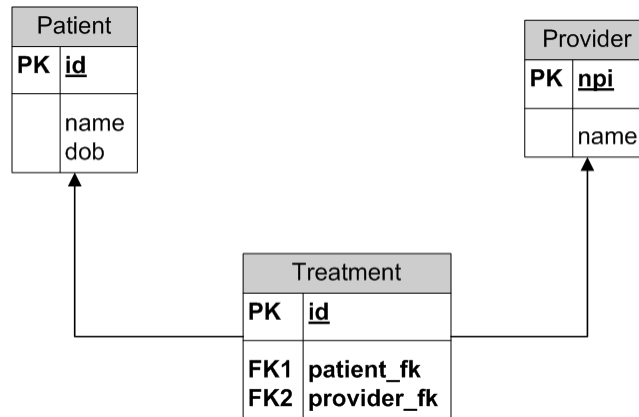
Example: E-R Diagram



151

151

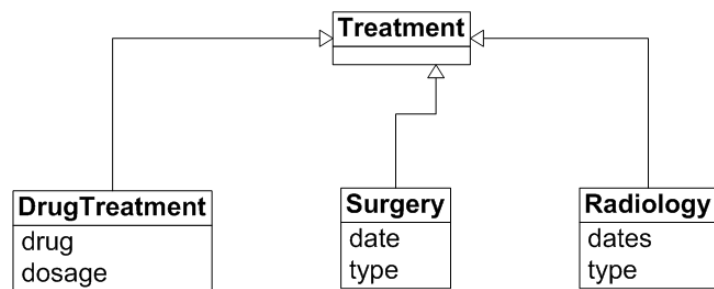
Example: E-R Diagram



152

152

Example: Forms of Treatment



153

153

Single Table Strategy

Treatment	
PK	<u>id</u>
	ttype patient provider drug-name drug-dosage surgery-type surgery-date radiology-type radiology-dates

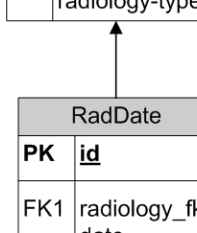
154

154

Single Table Strategy

Treatment	
PK	<u>id</u>
	ttype patient provider drug-name drug-dosage surgery-type surgery-date radiology-type

RadDate	
PK	<u>id</u>
FK1	radiology_fk date



155

155

Single Table Strategy

ID	TTYPE	PATIENT	PROVIDER	DRUG-NAME	DRUG-DOSAGE	SURGERY-DATE	RADIOLOGY-DATES
1234	D	8907	5643	Prednisone	20mg		
5678	S	9076	3412			4-02-2011	

156

156

Single Table Strategy

```
@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="TTYPE")
public abstract class Treatment
    implements Serializable {
    private UUID id;
    private String treatmentType;
    @Column(name="TTYPE", length=2)
    public String getTreatmentType() {
        return treatmentType;
    }
    ...
}
```

Treatment	
PK	<u>id</u>
	ttype
	patient
	provider
	drug-name
	drug-dosage
	surgery-type
	surgery-date
	radiology-type
	radiology-dates

157

157

Single Table Strategy

```
@Entity
@DiscriminatorValue("D")
public class DrugTreatment extends Treatment {
    private String drug;
    private int dosage;
    ...
}

@Entity
@DiscriminatorValue("S")
public class Surgery
    extends Treatment {
    private String type;
    private LocalDate date;
    ...
}
```

Treatment	
PK	<u>id</u>
	ttype patient provider drug-name drug-dosage surgery-type surgery-date radiology-type radiology-dates

158

158

Single Table Strategy

```
@Entity
@DiscriminatorValue("R")
public class Radiology extends Treatment {
    private String type;
    private LocalDate date;
    ...
}
```

Treatment	
PK	<u>id</u>
	ttype patient provider drug-name drug-dosage surgery-type surgery-date radiology-type radiology-dates

159

159

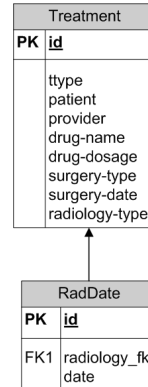
Single Table Strategy

```

@Entity
@DiscriminatorValue("R")
public class Radiology extends Treatment {
    private String type;
    @OneToMany
    private Collection<RadDate> dates;
    ...
}

@Entity
public class RadDate {
    private LocalDate date;
    ...
}

```



160

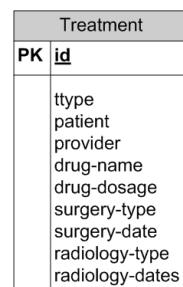
160

Single Table Strategy

```

@Entity
@DiscriminatorValue("R")
public class Radiology extends Treatment {
    private String type;
    @ElementCollection
    private Collection<LocalDate> dates;
    ...
}

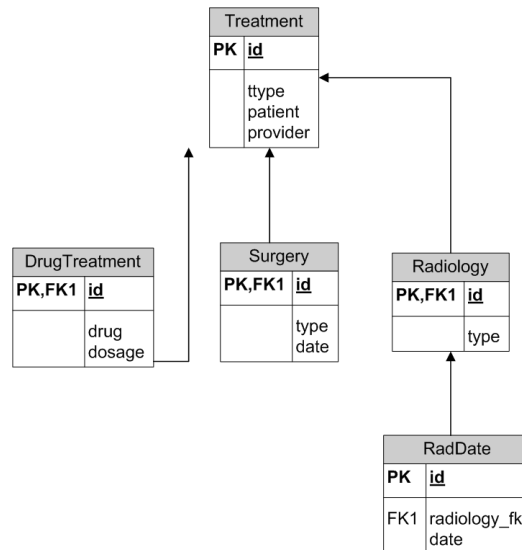
```



161

161

Joined Table Strategy



162

162

Joined Table Strategy

ID	TTYPE	PATIENT	PROVIDER
1234	D	8907	5643
5678	S	9076	3412

ID	DRUG-NAME	DRUG-DOSAGE
1234	Prednisone	20mg

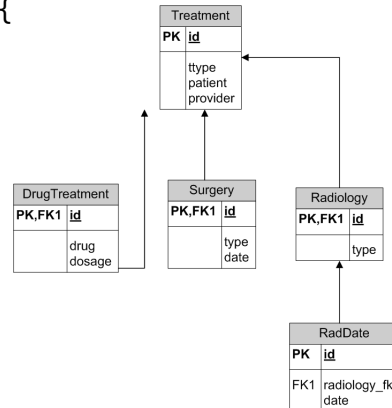
ID	SURGERY-DATE
5678	4-02-2011

163

163

Joined Table Strategy

```
@Entity
@Inheritance(strategy=InheritanceType.JOINED)
public abstract class Treatment
    implements Serializable {
    private UUID id;
    ...
}
```



164

Summary

- Single Table Strategy:
 - Best for reads and updates
 - Wasted space in the table
- Joined Table Strategy:
 - Cost of joins for queries

165

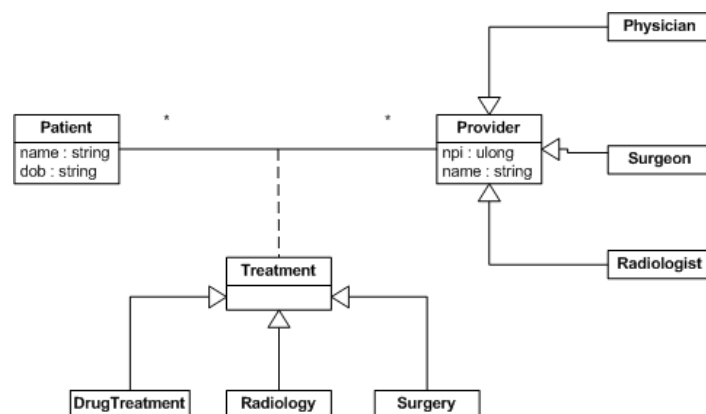
165

AGGREGATE PATTERNS

166

166

Example: UML Diagram



167

167

Visiting Treatments

```
Patient patient;  
Collection<Treatment> ts = patient.getTreatments();  
for (Treatment t : ts)  
    if (t instanceof DrugTreatment) {  
        DrugTreatment dt = (DrugTreatment)t;  
        ... t.drug ... t.dosage ...  
    } else if (t instanceof Surgery) {  
        Surgery s = (Surgery)t;  
        ... s.date ...  
    } else if (t instanceof Radiology) {  
        Radiology r = (Radiology)t;  
        ... r.dates ...  
    }  
}
```

168

168

Visiting Treatments

```
Patient patient;  
Collection<Treatment> ts = patient.getTreatments();  
for (Treatment t : ts) {  
    switch (t) {  
        case DrugTreatment dt ->  
            ... dt.drug ... dt.dosage ...  
        case Surgery s ->  
            ... s.date ...  
        case Radiology r ->  
            ... r.dates ...  
    }  
}
```

169

169

Visiting Treatments

- Problem: Violation of encapsulation
 - Aggregate pattern
- How do we:
 - Encapsulate treatments in patient aggregate?
 - Allow access to the treatments?
- Key patterns:
 - Visitor pattern

170

170

Visitor Pattern

- A “visitor” to a treatment must handle three possible cases
 - Drug treatment
 - Surgery
 - Radiology
- Represent as an interface:

```
public interface TreatmentVisitor {  
    public void visitDrugTreatment  
        (String drug, String dosage);  
    public void visitSurgery(LocalDate date);  
    public void visitRadiology(List<LocalDate> dates);  
}
```

171

171

Visitor Pattern

- A base class defines a visitor method signature:

```
public abstract class Treatment {  
    public abstract void visit(TreatmentVisitor v);  
    ...  
}
```
- A concrete subclass dispatches the appropriate case:

```
public class DrugTreatment extends Treatment {  
    @Override  
    public void visit(TreatmentVisitor v) {  
        v.visitDrugTreatment(this.drug, this.dosage);  
    }  
}
```

172

172

Visitor Pattern

- Patient aggregate dispatches visitor

```
@Entity  
public class Patient {  
    @OneToMany  
    private Collection<Treatment> treatments;  
    public void visitTreatments  
        (TreatmentVisitor v) {  
        for (Treatment t : treatments) t.visit(v);  
    }  
}
```

173

173

Aggregate Pattern

- Problem: Iteration over treatments should be client-controlled
 - Not aggregate-controlled
- Return a list of treatment identifiers
- Visit a treatment through patient API

174

174

Aggregate Pattern

- Patient aggregate returns list of treatment ids

```
@Entity
public class Patient {
    @OneToMany
    private Collection<Treatment> treatments;
    public List<UUID> getTreatmentIds() {
        ArrayList<UUID> ids = new ArrayList<>();
        for (Treatment t : treatments)
            ids.add(t.getId());
        return ids;
    }
}
```

175

175

Aggregate Pattern

- Visit a treatment entity through patient aggregate

```
@Entity
public class Patient {
    @OneToMany
    private Collection<Treatment> treatments;
    public void visitTreatment
        (long id, TreatmentVisitor v) {
        Treatment t = TreatmentDAO.get(id);
        t.visit(v);
    }
}
```

176

176